计算机组成原理 · CPU 数据通路和流水线 · 考点精讲与真题整合

408 各科大题知识点强化课 | 课程第 8 讲（分 P10） | 时长 3 小时 14 分 26 秒

本讲范围：（计）组原理第五章 CPU 的两个大题小节——**数据通路**（微操作序列、控制信号、单总线/多总线、能否合并到同一时钟）与**指令流水线**（五阶段、性能公式、流水段寄存器、三类冒险与解决）。开场他先定位了范围：「今天我们要学的话主要是第五章」`[00:01:05]`，第五章里「考大题的话只有流水线和数据通路」`[00:01:57]`。

读法建议：本讲是「知识点密、题量少」的一讲——他完整走了三道大题（2009 年数据通路大题、2022 年数据通路大题、2012 年流水线大题），另加一道 2023 年流水线选择题与四道课后题（24、25、28、29）。真正的价值在**§2 的公式与表格**（五阶段公式、三类指令在 EX/MEM 的差别、三类冒险的判定与解决）和**§3 的四条时序约束**。

一句话先记住：流水线判冒险不能只看「写后读」，还要看**中间隔了几条指令**（隔 ≥ 3 条就不冒险）；前面是 `load` 时，转发也救不了，**EX 必须排在 MEM 之后**。

目录

1 考情分析：范围、题量与他的学习法

2 核心概念：数据通路、控制信号、流水线

3 解题方法论：四条时序约束与写信号纪律

4 典型题型与模板：四道题的演算

5 统考真题与年份认定依据

6 易错点清单

7 考场速查卡

附 本资料的来源与边界

ASR 校正说明（读引用前先看这一段）

讲稿由本地 ASR 转写（同批讲稿口径为 Whisper），**这一讲的错字几乎全部集中在术语和缩写上**，同一段里简繁混杂。引用时已统一校正：**大体 → 大题**；**整体 / 真体 / 针铁 / 大铁 → 真题 / 大题**（转写里「整体大帖」「整体大铁」都是「真题大题」）；**主统原理 / 祖宗原理 / 组织原理 → 组成原理**；**驱指 / 趋指 / 曲子 / 取速 → 取指**；**异码 / 一码 / 义码 → 译码**；**战存器 / 战损器 / 站存器 → 暂存器**（正确术语是「暂存器」，转写多写成「战存器 / 战损器 / 站存器」，本资料统一写「暂存器」）；MAL / MA2 / M1R / mr / mcr → MAR；M2 / MD2 / MDL / M 第二 / 内二 → MDR；**爱二 / 艾尔 / 爱儿 / 二二 → IR**；**一码器 / 异码器 → 译码器**；**三太门 / 三泰文 → 三态门**；**单种线 / 多种线 / 总监 / 总键 → 单总线 / 多总线 / 总线**；**组设 / 阻涉 / 注射 → 阻塞**；**防写 → 仿写（照抄格式）**；**会边 / 汇变 → 汇编**；**十种周期 / 始终周期 / 时动作期 / 时农作企 → 时钟周期**；**二净之位 → 二进制位**；**十六禁止 / 十六经济 → 十六进制**；**漏的指令 / low 的指令 → load 指令**；**load 右人 → load-use**；**右翼 → 右移**；**副 513 → −513**；**质少 → 至少**；**指定 → 指令**；**指点 → 知识点**。

两处需要单独说明的校正：① **「开始」在这份讲稿里几乎都是「cache」**——依据是他讲 14 年那道题时说「就是开始相关的……里面问的都是典型的开始一些东西，**命中率**啊之类的」（`[02:39:02]`–`[02:39:12]`）；② **「R20」应为「R0」**——他同一段里也写作「R0」（「R0 直接连的就是这个总线」`[00:50:21]`），且经 2009 年题面核对，加法指令为 `ADD (R1), R0`（见 §附的核对说明）。

1 考情分析：范围、题量与他的学习法

1.1 本讲在课程里的位置

他开场交代：上一次课把**指令（寻址）**那一块「挑了几道题」讲完，「没讲的那些题目，当时也发了一些资料……可以对着之前发的资料，可以去看一下回放」（`[00:00:06]`–`[00:00:30]`）。今天进第五章。

> 「408 的大题尤其（计）组原理它其实比较集中：（计）组原理一本书其实一共七个章节……但是放到如果单纯来看大题的话，它其实很集中，这上面其实只有六七个小节有大题，1、2、3、4、5、6、7、8——相当整本书你只需要重点学习这**八个小节**就可以了，其他地方一般是只考选择。」（`[00:00:41]`–`[00:01:03]`）

> 「数据通路有**三道大题**，流水线是三（道）……（其中）有一道是 cache 的，并不是流水线的，相当流水线是有**两道大题**。」（`[00:01:13]`–`[00:01:22]`，⚠️ 转写作「流水线是三简易……这个简易是因为有一道是开始的」；「开始」= cache，见 ASR 说明）

1.2 他给的考频口径（全部出自他的口播）

考点	他的话（校正后）	出处
数据通路	「这种知识点这种知识点的大题叫数据通路，目前来说统考考过**三次大题**」	`[01:01:01]`–`[01:01:06]`
流水线难度	「流水线在 408 里面，尤其（计）组原理这门课，它算是一个**难点**」	`[01:37:42]`–`[01:37:48]`
流水线考频	「但是它虽然难，但是考试的**频率并不高**，就两道大题；而且待会儿在做大题的时候会发现，这两道大题其实也并**不是单纯的只考流水线**，他会考一些其他东西」	`[01:37:49]`–`[01:38:03]`
流水线的备考价值	「因为这一块还是**很容易出大题**的，正好也**很多年没考了**，它是很容易很适合出大题的」	`[02:38:44]`–`[02:38:48]`
组原难点分布	「（计）组原理这门课其实难的也就是**四五章**会难一点……就是补码呀、或者说指令、或者 CPU 这三个章节的大题会难一点」	`[02:59:39]`–`[02:59:59]`
收尾定调	「组成原理最最难的一部分已经过完了……一定要保证把**这个四五章的大题把它研究透**，最难的其实也就在这两章了」	`[03:13:30]`–`[03:13:49]`

他给本讲定的调子（背这四句就够）

① **大题要学三遍**：「408 的（真题）大题要学三遍了，一定要学三遍——考前的话学三遍，做真题之前先把它学两遍，然后真题再学一遍；如果做完真题还有时间，可以再学一遍，总共弄过四遍」（`[03:00:26]`–`[03:00:38]`）。

② **大题没有过程分**：「像 408 的大题你就直接写答案就行，没有什么过程分……你只要把答案写上去就可以，除非明确让你写计算过程」（`[01:09:08]`–`[01:09:25]`）。

③ **选择题要卡时间**：「在考试的话一定要控制选择题的时间，控制在这个 **70 分钟**左右做完，再长的话就不太合理了」（`[02:44:17]`–`[02:44:23]`）。

④ **别在一小问上卡住**：「千万不要在一小问卡（太久）」（`[01:22:01]`）；他讲 2022 年题第一问时专门说：「即便考试如果第一问不会写，其实跳过去写其他小问也都是没问题的」（`[01:21:57]`–`[01:22:01]`）。

课下要补的四道题（他明确布置）：「一个是一五年的两道数据通路的题目——数据通路和指令，一五年有两道题；其次还有一四年两道题（cache），相当有四道题可以留到课下」（`[03:13:11]`–`[03:13:25]`）。他同时交代课堂的取舍：「如果课堂上又讲知识点、又把每个大题全部都讲的话，这个是时间可能不够，所以我们就主要挑一些典型的去讲一下……课堂上应该我们能讲到 **70% 到 80%** 的这个题量」（`[01:24:36]`–`[01:25:01]`）。

2 核心概念：数据通路、控制信号、流水线

2.1 CPU 的两大部件与寄存器缩写（他说这块「纯粹记忆」）

「这个是要记一下的，就（运）CPU 分成**运算器**和**控制器**，运算器有哪些硬件、控制器有哪些硬件，这些东西是要记一下，包括每一个硬件的缩写——像算术逻辑单元是 **ALU**，暂存器一般是（⚠ 转写此处只剩「t 用 t 表示」，本资料不替他补这个缩写，见 §附⑥），累加器一般是用 **ACC**，通用寄存器的话一般是用 **GPR**，程序状态字寄存器叫 **PSW**（转写作「psw2」），所以它有时候也叫标志寄存器，有时候简写成 **FR**（转写作「F2」），都是同一个东西；然后控制器的话是有 **PC、IR、MAR、MDR**，这些缩写都是要记得，都是频繁会用到的。」（`[00:02:28]`–`[00:03:24]`，⚠ 他念「暂存器」的缩写时转写不清，本资料不替他写缩写；`TR` 处按教材习惯补，非他口述）

「第二个要记忆的点就是哪些是**可见的**、哪些是**不可见的**。可见的话有这几个：**通用寄存器、程序状态字、程序计数器、累加器**，这些是可见的；不可见的话是下面这几个：**MAR、MDR、IR、暂存器**，这几个是不可见的。」（`[00:05:22]`–`[00:05:41]`）

各寄存器用途（他说书上都有解释，「简单看一下」就行）：PC「存放**当前指令的地址**」；IR「放的是**当前正在执行的指令**」；MAR/MDR「放的是**内存的地址**和这个具体的**数据**」；PSW/FR「存放这些**标志位**的」；通用寄存器「就用来存放数据的」；ACC「可以存放 ALU 运算的结果」；暂存器「是用来**暂时存储**这个数据的，待会做（大）题都会用到」（`[00:04:14]`–`[00:05:10]`）。他给这一小节的定位：「第一小节就是把这些东西记一下就可以了……这边只有个别的（几道）选择题，到时候课下可以做一下就可以了，影响不大」（`[00:05:52]`–`[00:06:00]`）。

2.2 指令周期 / 机器周期 / 时钟周期，与取指阶段的数据流

> 「一条指令整个执行有一个概念叫**指令周期**，然后这个一个指令周期又是由若干的**机器周期**（组成）……这里要区分清楚几个概念：一个叫指令周期、一个叫机器周期、一个叫时钟周期，三个概念要区分开。但这个**指令周期是最长的**……机器周期的话是分阶段的，比如说取指令是一个阶段、执行是一个阶段，然后指令里面算出有效地址也是一个阶段；然后**时钟周期是最小的时间单位**，一个机器周期它可能也要消耗若干个时钟周期。」（`[00:06:18]`–`[00:07:33]`）

```
【取指阶段的四个微操作（他说「这几个步骤是经常要会写的」）
PC → MAR              送指令地址
M(MAR) → MDR          「根据 MAR 去访问内存，取出来数据放到 MDR」（读内存）
MDR → IR              「再把 MDR 的数据送到 IR」
(PC) + 1 → PC         「然后当然 PC 最后还要自动加一」
出处：[00:07:40]–[00:08:04]
```

他把这条链讲得很细：「你要从内存里面取东西，你要先把 PC（的内容）——你这个 PC 放的就是指令的地址——先把指令的地址送到 MAR，MAR 再送到地址线，然后送到地址线之后由这个地址线再去访问内存，访问内存把数据读到数据线，再送到 MDR；送到 MDR 之后，如果是取指阶段，他还要送到这个 IR」（`[00:08:18]`–`[00:08:50]`）。他还补了图上一个省略的细节：「这个地址线其实是好几根的，每一根只能传送一个二进制位……这个图画得比较精简，现在地址线它就画（成）一根，相当这一根背后其实有若干根构成」（`[00:11:12]`–`[00:11:46]`）。

2.3 数据通路与控制信号：这一小节最容易考的写法

> 「数据在指令执行过程中经过的**路径**叫**数据通路**……像这个 ALU、一些通用寄存器、状态寄存器这些东西，都是指令执行时数据流进的部件，都属于数据通路的一部分。然后数据通路描述的是**信息从哪里开始、中间经过哪些硬件、最后被送到哪里**。」（`[00:12:44]`–`[00:13:09]`）

> 「带箭头的话（是）控制信号……**in 的话表示写入，out 表示输出**；**MDRin** 就是说内部总线上的数据写入 MDR；然后 **MDRout** 是 MDR 的数据送到内部总线。」（`[00:25:35]`–`[00:26:14]`）

```
【他讲的第一个例子：把 PC 的内容送到 MAR】
功能（左）            控制信号（右）
(PC) → MAR             PCout = 1      ←「等于 1 那说明 PC 的数据是往外流的」
                       MARin = 1      ←「外面这个 MAR 是往里面接收数据的」
「所以 MAR 是 in、PC 是 out，这种东西就叫做控制信号。」[00:14:19]–[00:14:21]
「今后的话碰到的题目要知道控制信号是什么东西，有人会让你写，这个真题里面经常会让写一些控制信号。」[00:14:23]–[00:14:28]
```

他点名的第一个陷阱：同名信号看方向

「你在读这句话：**MDRinE** 是说数据从**数据总线**传入到 MDR……所以 MDRinE 是**和内存打交道**的；然后 **MDRin** 的话是说从（CPU 内部）总线到 MDR，从 CPU 的总线到 MDR，这个叫 MDRin。要把这两个区分开：一个是从上面内存这个方向过来，一个是从 CPU 的总线过来。」（`[00:32:56]`–`[00:33:32]`，⚠️ 转写作「MDR 印役」，按上下文应为 `MDRinE`）

两套总线要分清：「上面这个控制总线、数据总线、地址总线，叫做**系统总线**，系统总线是用来这个内存和 CPU 之间交换数据的；那具体 CPU 内部的这些硬件都是通过 **CPU 内部总线**来交换数据」（`[00:33:37]`–`[00:33:51]`）。

和 2022 年题的差别：「2009 年这个（大）题……它给了两组控制信号，一个是从内存过来的、一个是从内部总线过来的，是给了两组；然后下面这个题目它只给了一组，都是从内部总线过来的，要注意（真题）」（`[01:02:17]`–`[01:02:44]`）。

还有一个他单独强调的题面条件：「这里要注意一句话叫 **MAR 一直处于使能状态**……这个使能状态就是说 MAR 的数据一直没有丢失——就是你之前 R1 已经把这个地址送到 MAR 了，中间 MAR 没有被更改成其他数据，那说明这个数据就一直还在，你下次就不用重新把 R1 的数据送到 MAR」（`[00:43:14]`–`[00:43:49]`）。

2.4 三个硬件：译码器 / 多路选择器 / 三态门

```
译码器（多对多）：「比方说这边有三个二进制（位），它一共能对应八种状态，你可以
  把它映射到八种状态」—题里就是「把四个东西（映射）到 16 种状态」（2022 年第三问）
多路选择器：「从这两个数据只选一个把它输出……传过来一个控制信号，用来控制把这两个
  其中的一个把它输出」（他让你回忆加法器电路图里那个多路选择器）
三态门：「信号等于零他就传不了（把这个堵住了），如果信号等于 1 的话，他就能把数据
  顺利传过去」—所以三态门就是用来控制传送数据的
出处：00:14:53–00:17:19、01:06:40
```

2.5 单总线 vs 多总线：一条最关键的时序规则

「**单总线的话一个时钟只能传一个数据**，这句话要注意，很关键的一句话……现在你看这个 PC 到 MAR 是需要通过这个单总线去传，MAR 到 MDR 也要通过单总线去传，所以它们**一定要分在两个时钟周期**。」（`[00:18:57]`–`[00:19:27]`）

「多总线的话**一个时钟是可以传送多个数据**的。但是在 408 里面 80% 或者目前的题目基本都是（单）总线的……总线只有这么一根。」（`[00:20:20]`–`[00:20:39]`）

系统总线的三个缩写他也要求记：「CB 是控制总线，DB 是数据总线，AB 是地址总线」；读写方向看箭头——「往下面是读、往上面是写（写回到内存）……最后控制线再传一个读写命令」（`[00:23:07]`–`[00:24:11]`）。

2.6 五阶段流水线：公式与流水段寄存器

> 「一个指令的执行分成了**五个阶段**：**取指、译码、读寄存器、执行、访存、写回**，这些缩写要记一下」（`[01:38:26]`，他念的是六件事，其中「译码 + 读寄存器」同属第二阶段：「这个注意第二个阶段，除了对操作码进行译码，还有的话就从寄存器读取数据」`[01:38:43]`–`[01:38:53]`）。

> **本讲唯一给出的性能公式（背下来）**
>
> **K 级流水线执行 N 条指令的时间 = (K + N − 1) × T**（T 为一个时钟周期）。
> 他的两种理解：「第一种理解，第一条指令是需要执行 **K 个阶段**，那后面的 **N−1 条**……每一个阶段比方说是一个时钟周期，第一条需要 **K·T**，后面的 N−1 条每次只需要一个 T 就能执行了，所以总共是 **(K + N − 1)T**」；「还有第二种理解，就是每一个时钟周期都会取一条指令，那你把 N 条都取完是 **N·T**，最后一条指令取完之后还要经历 **K−1** 个时间，这样也能算出来是 (N + K − 1)T」。（`[01:40:02]`–`[01:41:51]`）

> 「每个流水段后面都要有一个**流水段寄存器**，用于**锁存本阶段的数据**，然后用来保证本阶段的执行结果在**下个时钟周期供给下一个流水段使用**。」（`[01:42:16]`–`[01:42:32]`，他引的是书上的话）

> 【为什么必须有流水段寄存器（他举的反例）】
> 第一条指令取指完之后「会把这个指令放到流水段寄存器这里」；
> 等到你去取第二条指令，「因为你完成取指阶段就意味着要把指令送到 IR 这个寄存器，
> 那这时候你就不用担心译码，因为你译码的话就不用担心 IR 里面被覆盖了」。
> 没有它的话：「等到第二个阶段你去取第二条的时候……第二条指令把（指令）放到 IR，
> 你一旦放到 IR，那这个译码的时候很有可能就会拿到这个下面的（新指令），
> 按理说它译码应该是拿上面的（老指令）」→ 译码就错了。 [01:43:23]–[01:46:23]
> 【课后题 28 题（262 页）：时钟周期怎么取】
> 「它先给出了每一个流水线每一个阶段的时间，然后最后它问你时钟周期，你不能只把
> 这个阶段的事情执行完就行了，因为每一个阶段它把数据完成之后还要专门花一个时间
> 把数据放到这个流水段寄存器，所以每一个阶段还要加上一个 20……最后时钟周期就
> 应该选这个**最大的**，然后加上这个流水段寄存器，就是 **100**。」 [01:47:38]–[01:48:28]
> ⚠ 他只报出「延迟 20」和结果「100」，没有把各段的具体时间念出来（书上的表格），
> 本资料不替他补各段数值。

2.7 三类指令在 EX / MEM 阶段的差别（他说这张表「一定要记」）

> 「这个表格要记一下，这个很多人可能看书的时候没有意识到这个表格的重要性，有这些小节的很多东西都要结合这个表格去做的，5.2（节）是一定要记一下的。」（`[01:50:01]`–`[01:50:13]`）

指令类别	EX 阶段	MEM 阶段	WB 阶段
计算类（ALU）	运算	**空的，什么都不做**	写回寄存器
访存类（取/存）	**算有效地址**	**访存**（读或写）	「将内存里面读出的数据写到寄存器」
转移类	**算出目的地址**	**把算出来的目的地址送到 PC**	—

他的原话：「如果是（计算类）指令，它的 M 阶段是空的，什么都没干，画了个斜线什么都没有；如果是取存指令，注意它的 **EX 阶段**，它是用来算一个**有效地址**，然后在 **MEM 阶段访存**，访存之后注意还有一个过程，它是写到寄存器（WB）」（`[01:50:37]`–`[01:51:44]`）。「转移指令的话就是说在这个计算阶段它是先算出来这个目的地址，然后它这个 MEM 阶段是把算出来的目的地址送回到 PC」（`[01:55:30]`–`[01:56:01]`）。

他为什么要分这三类——因为流水线小节的**访存和计算是分开的**：「在之前的学习过程中，我可能一条指令是这样的：他把 10 号地址的 x 取出来，把 20 号地址的 y 取出来，再把两个进行相加……现在如果学到流水线这小节，改了就不是这样了……原来一条指令就能干完，现在分成了三条」（`[01:52:47]`–`[01:53:26]`）。

2.8 三类冒险：判定与解决（本小节的重点）

> 「这个流水线的冒险和处理，这个才是本小节的重点。」（`[01:49:28]`–`[01:49:34]`）

冒险	判定（他的话）	解决（他的话）
结构冒险 （资源冲突）	「不同指令在**同一时刻用了同一个功能部件**……这个叫资源冲突」（`[01:56:34]`–`[01:56:53]`）：取指的 IF 访存、访存类的 MEM 也访存，「因为指令和数据都是在内存的」（`[01:57:47]`）	① 排流水线时把 IF 往后推：「所以你会发现这个题干上就把这个 IF 往后推迟了，所以你得知道为什么这里推迟了，因为要避免 IF 和这个 MEM 放一块」（`[01:58:48]`–`[01:59:01]`）；② 「所以一般会把这个**指令 cache 和数据 cache 分开**，因为你取指令和取数据都要访问内存」（`[01:59:49]`–`[02:00:03]`）
数据冒险 （数据相关）	「它的原因是这个：**后面指令会用到前面指令的结果，但是前面指令的结果没有产生**……主要就是这个**写后读**」（`[02:04:13]`–`[02:04:24]`）。原因是流水线让「上一条指令还没有全执行完，下一条其实就开始了」（`[02:05:41]`）	① **硬件阻塞**：「你让取指阶段多往后等一会」（`[02:07:14]`–`[02:07:25]`）；② **插入空操作**：「中间给它插入三条空操作指令……凡是指令就要经过取指周期、这个译码这些过程」（`[02:09:44]`–`[02:10:41]`）；③ **数据旁路 / 转发**：「不等前一条指令把计算结果写回……直接把生成的中间数据转发到 ALU 的输入端」（`[02:11:12]`–`[02:11:29]`）；④ **load-use = 延迟 + 转发**（见下）
控制冒险	「因为指令是顺序执行的，所以控制冒险往往会和**转移指令**有关」（`[02:19:02]`–`[02:19:21]`）：「如果（if）条件不成立，它这个程序就直接跳转执行了，那直接跳转执行就意味着……后面其实本身没必要执行了」（`[02:20:12]`–`[02:20:25]`）	① **尽早生成转移的目标地址**：「你到底这个程序要转移到哪里，要尽早把它算出来」（`[02:20:39]`–`[02:20:51]`）；② 排图时「下一条指令的 IF 一定要在（上一条）转移指令的 MEM 之后」（`[02:21:37]`–`[02:21:43]`）

load-use：为什么转发也救不了

「它的意思是说 `load` 就是**访问内存的指令**……如果前面指令是访问内存，它其实它的数据在 **MEM 的时候才取完**……如果是一条 `load` 的指令，它的数据不是在 EX 阶段算完的、不是在 EX 阶段就拿到的，是在**访问内存之后**才拿到的，所以你原来的（转发）策略它是行不通的，所以它必须保证这个 **EX 在 MEM 之后**，因为访存指令它在 MEM 阶段才拿到数据、再传给 EX。所以它的核心这个方法叫做**延迟加转发**。」（`[02:13:30]`–`[02:15:01]`）

他还提醒书上印错了：「当然这个书上这边也是 add 和 sub 就错了，注意改一下，这里应该是 `load`，下面应该是 `add`，书上的印刷错了」（`[02:13:42]`–`[02:13:55]`）。

他还给了这一小节的阅读建议：「学这个流水线，书上文字其实有点乱，但是书上这几幅**表格都要能看明白**，就足够了」（`[02:22:58]`–`[02:23:08]`）。讲完三类冒险他又回头总结了一遍：「东西有点（琐碎）……首先最（前面）讲了这个不同类型的指令，像什么计算类、（访存）类、转移类的，不同类型的指令它 EX 阶段和 MEM 阶段的区别；然后再往后讲了这个结构冒险……经常的一个解决办法就是指令 cache 和数据 cache 分离……再往后就是数据冒险，数据冒险是后面指令会用到前面指令的结果，就相当于是写后读」（`[02:15:44]`–`[02:16:46]`）。

3 解题方法论：四条时序约束与写信号纪律

3.1 写微操作序列 / 控制信号的三条纪律

```
纪律一：先写功能、再写控制信号；题目要控制信号就一定要写控制信号。
  「做题目的时候一定要审题……人家让写的是控制信号，不要写这种（微）操作。
   当然你可以先写出来作为辅助也可以，但是一定要最后还是要把控制信号写出来。」[01:06:52]–[01:07:10]

纪律二：凡是和内存打交道，必须借助 MAR 和 MDR。
  「然后凡是和内存打交道，必须借助 MAR 和 MDR。」[00:43:09]–[00:43:14]

纪律三：ALU 不存储，所以运算前要把一个操作数放暂存器、运算后要放回暂存器/ACC。
  「ALUs 这个硬件是没有存储数据的功能的，它是不能存储数据……如果你没有这个暂存器，
   它运算完直接传到总线上，总线可能又传到输入端，会互相干扰，这个就乱掉了。」[00:42:01]–[00:42:33]
  注：他在这一讲里把这个暂存器写作「A」；2022 年题面写作暂存器 Y 和 Z（见 §附的题面核对）。
```

3.2 数据通路题：怎么判断两步能不能合并到同一个时钟

合并的唯一判据：两步走的是不是同一条总线

「一个是 R0（的数据）到 A，它用的是 **CPU 总线**；还有一个是从内存送到 MDR……**它用的是系统总线**。两个是**互不干扰**的，所以可以同时执行。」（`[00:45:53]`–`[00:46:20]`）

反之，都走 CPU 内部总线的两步就不能合并——依据单总线规则「一个时钟只能传一个数据」（`[00:19:00]`）。

还要看题目问什么：「两种写法都对，因为这个题目它并没有要求让你用最少的时钟；如果题目里明确要求需要你用这个最少的时钟，那这时候你必须写（优化）这种」（`[00:47:49]`–`[00:48:10]`）。

3.3 四条时序约束（排流水线图时逐个套）

#	约束	他的话与适用条件
①	**ID 在 WB 之后**	「如果没有采用转发技术，那你只要如果发生写后读，你保证这个**读在写之后**就可以了」（`[02:45:53]`–`[02:46:02]`；ID 是读寄存器、WB 是写寄存器）
②	**EX 在 EX 之后**	「转发的话它是保证 **EX 在 EX 之后**」「但凡用转发的话，你只要保证 EX 在 EX 之后就行，下条的 EX 在上条的 EX 之后就可以了」（`[02:45:58]`、`[02:29:44]`–`[02:29:53]`）
③	**EX 在 MEM 之后（上条是 load）**	「因为第二条是一条 `load` 的指令……因为 `load` 的指令它在 M 阶段才会拿到数据，所以你的 EX 应该放在 M 之后」（`[02:30:53]`–`[02:31:30]`）
④	**IF 在 ID 之后；上条是转移指令时 IF 在 MEM 之后**	「这个 IF 一定要在上条的 ID 下面……最保守也应该在 ID 下面」（`[02:57:22]`–`[02:57:32]`）；「下一条指令的 IF 一定要在上一条——如果上一条是转移指令——一定要在转移指令的 MEM 之后，因为在 MEM 这个阶段它才会把这个转移的目的地址送到 PC」（`[02:21:46]`–`[02:22:01]`）

> 「做题目的时候，要靠这些约束一条条卡：正常来说应该这样去画，但是因为前一条是转移指令，转移指令你在（EX）才会把这个地址算出来，并且在 EX 阶段把转移的地址算出来、在 MEM 阶段把它放到 PC，所以你取指的话肯定要放在（MEM 之后）——只有先放到 PC 里面之后才能去取指。」（`[02:22:35]`–`[02:22:53]`，⚠️ 这一段他口述里 EX/MEM 交叉说了两遍，最终口径是：**EX 算目的地址、MEM 写回 PC**）

3.4 判数据冒险：不能只看「写后读」，要看间隔

> 「但是千万做题目的时候，不要单纯只通过写后读去判断……如果第一条指令比方说写回 R1……如果第五条指令是读 R1，所以这样的话依然是写后读，第一条是写、第五条读，但是因为中间正好间隔了三条指令，所以他的读刚好已经在写之后了。**这并不是说写后读一定是有数据冒险的**：如果写后读它中间间隔的指令刚好是三个那就没问题；如果小于三个，如果是写后读就相当会有数据冒险，**如果大于等于三那就没有**。」（`[02:35:11]`–`[02:36:14]`）

这条规则的原理就是 §2.6 的五阶段：写在第 5 拍（WB）、读在第 2 拍（ID），差 3 拍，所以隔 ≥ 3 条指令时读自然晚于写。

3.5 时间计算与「至少」这两个字

① 问「时钟周期数量」→ 只算 (K + N − 1)，不要再乘 T。
「当然这个题目注意（真）题，它没有问你需要的时间，它只问了时钟周期的数量……
就不用乘以 T 了，你乘以 T 算出来的是一个具体的时间。」[02:52:33]–[02:52:55]

② 问「至少」→ 一定存在多种方案，把更优的那种找出来再答。
「所以碰到这个『至少』的话……你如果写出来一个，一定要考虑一下有没有错误，
或者有没有这个更简便的方法、或者更优化的这种方案。因为他说至少，
一般可能是有多种情况，你写的可能就不是那个至少的情况。」[03:03:11]–[03:03:30]
★ 他的实例：同一条语句两种指令序列，数出来一种 18 个时钟、一种 17 个，
他专门提醒「人家问至少，你回答 17、回答 18 肯定就（错）了」[03:11:02]–[03:11:21]

4 典型题型与模板：四道题的演算

4.1 2009 数据通路大题：`ADD (R1), R0` 的执行阶段

题面（他念的要点）：「它说计算机（字）长（16 位）……这个（字）长用不到」「像这些位数其实是没有用的」；「他说控制信号为一表示有效、为零表示无效」（`[00:32:16]`–`[00:32:41]`）。指令功能的读法他讲得很细：

> 「`R1` 的括号（是地址），那说明 R1 的内容是地址，就 R1 这个寄存器里面的东西是一个地址，那说明它用的是**寄存器间接寻址**……然后 R0 是直接写的一个寄存器编号，就是**寄存器（直接）寻址**。」（`[00:34:33]`–`[00:34:58]`）
> 「加了两个括号的意思就是先把 R1 的内容取出来——是一个内存地址，有了这个内存地址再加一个括号，所以把内存里面的东西取出来」；「如果你直接写回到 R1，那说明你写回到寄存器里面；（但）你写回 R1 的括号，那说明你写回的那个地址也是 R1 这个寄存器所指向的东西，说明你写回的地方应该是**内存**，不是寄存器」。
> （`[00:35:12]`–`[00:36:10]`）

【取指阶段（题里已给，他复述了一遍；他写的节拍号是 C1、C2 这种，转写里不全，
下面只保留先后顺序）
① (PC) → MAR　　PCout、MARin
② M(MAR) → MDR；(PC) + 1 → PC　　读内存信号 + MDRinE（往内存方向）+ PC 加一
③ (MDR) → IR　　MDRout（「MDR 肯定是往 CPU 这个方向」
「所以应该是普通的 out，不要写错了」）[00:36:44]–[00:38:04]

⚠ 他板书时的节拍编号在转写里不全（他后面把执行阶段的第一拍叫「第 5 步」，
又提到「C7、C8、C9」）；本资料只保留先后顺序，不还原节拍号。

【执行阶段 · 写法一（先把 R0 放进暂存器 A）—— 原答案，6 个时钟周期】
1 (R0) → A　　算术逻辑单元的一端先占住一个数
2 (R1) → MAR　　「R1 这里是寄存器间接寻址，要去内存里面拿数据的，
凡是从内存拿数据你必须先把这个地址送到 MAR」[00:40:51]–[00:41:05]
3 M(MAR) → MDR　　读内存
4 (A) + (MDR) → AC　　「第二个数据在 MDR 里面，接下来就可以运算了，直接 A 加 MDR」[00:41:51]–[00:41:55]
5 (AC) → MDR
6 (MDR) → M　　写回 R1 指向的内存单元

【写法一的优化版，5 个时钟周期】把第 1 步和第 3 步合并：
「R0 到 A 和从内存送到 MDR……一个是 R0 的数据到 A，它用的是 CPU 总线；
还有一个是从内存送到 MDR，它用的是系统总线，两个是互不干扰的，所以可以同时执行。」[00:45:53]–[00:46:20]

【写法二（答案也给了）：先把 R1 指向的内存数取到 MDR，再送 A】—— 6 个时钟周期、无法优化
「先把 R1 的数据取出来送到 MAR，然后接下来之后再把 MAR 的数据——再从内存取出来
送到 MDR，然后你把 MDR 的数据放到 A（暂存器）……这时候 A 里面放的是第二个数，
那第一个数 R0 是可以直接用的，因为 R0 它直接连的就是这个总线。」[00:49:26]–[00:50:24]
「如果你写成这种写法，这个就优化不了了……因为首先这两步本身就有顺序要求：
你要先到 MAR，才能 MAR 到 MDR；先到 MDR，MDR 才能到 A。」[00:51:22]–[00:51:59]
「相当这个题并不是四种写法，最后只有**三种写法**……这个题目三种写法都行。」[00:52:01]、[00:52:39]

这道题他还留了话：「这个 09 年的（题）就不讲了……相对来说这个题目还是要简单一点，看答案应该就没有太大问题」，后来又改口「看一下这个也看一下吧」（`[00:25:03]`、`[00:26:49]`）——所以本讲里 09 年题**只讲了执行阶段的微操作与控制信号**。

4.2 2022 数据通路大题：五个小问（他先讲后四问，最后回头讲第一问）

他的讲解顺序有理由：「这个题目我们讲的时候最后再讲第一问——因为第一问其实和这个题目没啥关系，就是第二章的一个知识点，最后再讲，就趁着刚讲完数据通路先把后面几个小问讲了」（`[01:00:36]`–`[01:00:56]`）。

【第二问】为什么要设置暂存器？
「回答出来 ALU 没有存储功能」即可，两点理由：①「要同时处理两个数据，要先把一个暂存」；②「算完的结果直接就传到总线上了，很有可能会再次当作 ALU 的输入端，会有一些干扰」。 [01:02:56]–[01:03:41]
【第三问·前半】rs 和 rd 最多能表示多少个寄存器？
「rs 和 rd 它的意思（是）都只有四位……那四位编号的话能表示 0 到 15，相当 0 到 15 那能表示 **16 个寄存器**。」
★ 他补的通用结论：「如果告诉你 16 个寄存器，那就是寄存器编号需要四位；如果告诉你寄存器编号四位，那就是能表示 16 个寄存器，**互相推的**。」 [01:04:19]–[01:05:02]
【第三问·后半】rs 和 rd 来自哪个寄存器？
「寄存器编号肯定来自于指令的地址（码）……因为指令有操作码和地址码，如果是寄存器寻址，他就在地址码里面提供一些寄存器的编号」→ 来自 **IR**。[01:05:20]–[01:05:46]
【第三问·第三小问】GPRs 内部有地址译码器和多路选择器，选哪个？
「你这边一共有 16 个寄存器……你通过输入的这 4 个二进制位来映射到右边的这个 16 个寄存器中的其中一个，这个是多对多，所以应该是这个地址译码器。」
（译码器与多路选择器的区别见 §2.4） [01:06:08]–[01:06:44]
【第四问·控制信号序列（取指阶段，不考虑 PC 增量）】★ 这里有一个他点名的陷阱
① (PC) → MAR　　PCout、MARin
② 读内存 → MDR　　读信号（他提醒：「控制信号是有读信号的」）
③ (MDR) → IR　　MDRout、IRin
陷阱：「有的人上来会写一个读信号，还会写一个 MDRin，这时候就有问题了——你这个 MDRin 应该是从内存过来的数据……但是在这道题里 MDRin 指的是从内部总线过来的，不要搞混了，所以这个 in 一定不能写」，因为题目只给了内部总线这一组 MDRin；2009 年题里那条「从内存过来」的 in（MDRinE）才写。 [01:07:29]–[01:09:04]
【第四问·时钟周期数】
「发出（读）命令到读出数据需要 5 个时钟周期……因为你现在是访问内存，速度要慢点；前面 PC 到 MAR 和 MDR（到）IR，它都是寄存器之间来传数据的，都要很快，寄存器这边传数据都是一个时钟就够了」，所以**总共 7 个时钟周期**。 [01:09:33]–[01:10:13]
【第五问】控制信号由谁产生、根据什么产生？
「控制信号是由**控制单元**，就 CU 产生的……这个 CU 是根据什么产生的，是根据**操作码**产生的。」他顺手拆了 IR 的两半：「其实严格意义上来说就是把他的**操作码**送到 CU，**地址码**应该送到下面的寄存器这里——操作码是用来产生控制信号的，地址码是用来取数据的。」 [01:10:13]–[01:11:39]
「哪些寄存器会连到该部件的输入端」→ 他答两个：
① 指令寄存器（送操作码）；②「标志寄存器也要（连）」——因为标志寄存器里「会存放有一些溢出位等等……不进位、借位，还有零标志」，做条件判断要用。 [01:11:46]–[01:13:35]
【第一问】SF 与 OF 的逻辑表达式（和本题无关，是第二章的知识点）
SF：「这个 SF 是符号标志……符号位你就直接等于 F15 就可以了，因为你算完的最终这个结果 F15 就已经是最高位了。」 [01:16:03]–[01:16:57]
OF（加法）：「正加正等于负就说明溢出了；如果是负加负等于正，那也说明溢出了。」
写法：乘法（·）表示与运算、加法（+）表示或运算、「加一杠」表示取非。 [01:17:43]–[01:19:05]
$OF = (\neg A15 \cdot \neg B15 \cdot F15) + (A15 \cdot B15 \cdot \neg F15)$
OF（减法）：「正减负如果等于负是溢出了；如果是负减正等于正也是溢出的。」
$OF = (\neg A15 \cdot B15 \cdot F15) + (A15 \cdot \neg B15 \cdot \neg F15)$
★ 他的判溢出三法：「第一种是根据表示范围……第二种方法就是根据（原始数据的）符号和结果的符号……第三种方式就是根据（最高）数据位进位和符号位进位。但这个题目是根据符号位做的。」 [01:17:15]–[01:17:43]

⚠ 上面的两个逻辑式是与运算优先的缩写写法，与题面结构一致（加法那一支对应「正 + 正 = 负」，减法那一支对应「正 − 负 = 负」「负 − 正 = 正」）；他讲的时候是分两支口述的，没有把整式一次念完。这两问已与公开的 2022 年题面相互印证（见 §附）。

4.3 2012 流水线大题：四个小问

题面前提（他念的）：16 位计算机、有符号整数用补码表示、指令 cache 和数据 cache 分离、采用五阶段流水线、「这里是没有采用转发技术」（[02:45:43]–[02:45:53]）。他还提醒题干很长：「通过这个题的话你能看出来这个题干特别长，如果放到这个纸上起码要占一整页 A4 纸，可能还不够」（[02:44:02]–[02:44:14]）。

【第一问·−513 的补码与移位（第二章的考点）】
「相当于是这个 16 位计算机它指的是**机器字长**，机器字长一般是等于 ALU 的位数
和寄存器的位数……然后正常来说这个 int 的话它是一个 32 位，有（符号）的类型是
32 位，所以你把它放到寄存器，他只会保留（低）16 位。」 [02:46:18]–[02:47:07]
「（−513）你先写出这个原码，然后（写成）−（512+1），512 = 2 的 9 次方……
中间都是 0，这里最好每四位写成一组」→ 求补码：「最右边的 1 不动，
剩下的全部取反，中间本来是全 0 的，现在变成全 1 了」→ 16 位结果 **FDFFH**。
（他先在 32 位下写成 FF、FDF… 又改口：「因为这个寄存器是 16 位，你把它放到寄存器
只会保留（低）16 位，所以现在寄存器里面的内容是 FDFF」） [02:47:15]–[02:48:34]
「他要把这个东西（SHR）就是右移，通过看上面也能看出来 SHR 是右移、算术右移，
相当于把它除掉。计算机在移位的时候注意，本来是 F、F、（F）D、F、F，计算机在移位
的时候，**符号位是不参与移位的**，所以先把（符号位）保留……
如果是补码、如果是负数的话，中间应该补一个 1（空位补符号位）」 → **FEFFH**。
★ 他在这一问中途算错过一次（先写成 FFDF、算出 FFEF），随后自己纠正：
「这里算错了……这里的话应该是（D）后面还有两个 F」「F、D、F、F……FDFF」，
最终给出「移位完之后就是一个 FE、FF」。 [02:48:41]–[02:51:54]
⚠ 指令助记符转写作「sh2」，经 2012 年题面核对为 SHR R1；在「有符号、补码」
的语境下按算术右移处理（符号位不动、空位补符号），与他给的 FEFF 一致。
【第二问·连续 4 条指令的时钟周期数】
「每一个流水线它是五个阶段，你要执行四条指令，K 级流水线执行 N 条指令，
那就是（K + N − 1)T……」→ 5 + 4 − 1 = **8 个时钟周期**（他只问数量，不乘 T）。 [02:52:09]–[02:53:01]
【第三问·把 X = A + B 写成指令序列 + 执行过程示意图】★ 这一问考「仿写」
「先把 A 取出来放到 R1，再把 B 取出来放到 R2……再把两个相加」
「因为它这个取数和访存是分开的：如果你要采用流水线，它会把取数和访存分开，
不是说一步到位的。」 [02:53:29]–[02:54:07]
他解释两处阻塞：
· I3 的 ID 为什么停：「第三条指令几乎用到（R1）也会用到（R2）……因为前面是
写回 R1 和写回 R2，所以这个第三条应该 ID 在前面两条 WB 之后。」 [02:55:33]–[02:55:59]
· IF 为什么停：「如果你后面又去取下一条，相当于你又取了一条新的指令，
又把它放到了流水段寄存器，所以上条其实还没有来得及（译码）……
这样的话这个 ID 去译码的话，其实拿这个他拿到的指令就拿成了下一条的了。
所以 IF 一定要放在 ID 下面」→ 结论：「IF 应该在 ID 下面，你也可以更靠后，
但是起码在 ID 下面。」 [02:56:41]–[02:58:22]
【第四问·X = X×2 + A 至少需要多少时钟周期】★ 本讲最难、也最容易丢分的一问
新指令：x 乘 2 再加 a；x 和 a 都是无符号数，内存地址是 x( )、a( )。
「它问的是**至少**……这个题目有难度、是有坑的，很容易做错。」 [02:58:58]–[02:59:22]
方案一（取完 x 立刻左移）：取 x→R1；SHL R1（乘 2，写回 R1）；取 a→R2；
(R1)+(R2)→R2；R2 存回 x → 他数出来 **18 个时钟**
方案二（取完 x 不急着移位，先取 a）：取 x→R1；取 a→R2；R1 乘 2 写回 R1；
(R1)+(R2)→R2；R2 存回 x → 他数出来 **17 个时钟**
「但是人家问至少，你回答 17、回答 18 肯定就（不）对了，因为 408 又没有什么
过程分，所以这个一定要注意：他问了至少，一定要好好分析一下。」 [03:10:19]–[03:11:21]
「上面这种写法是 18 个时钟，下面这种写法 17 个，至少的话就选下面这个。」
「指令上面是时钟，把那 5 个阶段写成 IF、ID 这些东西。」 [03:11:34]–[03:12:06]

4.4 2023 流水线选择题：转发 + 硬件阻塞之后还剩谁被阻塞

> 「它是二、三年的一个（真）题，五阶段流水线，五个阶段……这个题目和之前的题目不太一样，它专门说了如果**采用了转发技术、又采用了硬件阻塞**去处理这个控制冒险——采用转发处理数据冒险、采用硬件阻塞处理控制冒险——然后问你依然发生阻塞的（指令）。」（ [02:26:59]、[02:28:22]–[02:28:46] ）

他复述的四条指令（寄存器名按转写照录，⚠️ 转写把它们写成 S1/S0/S2/S3/T2，疑为 R 系列寄存器）：
I1: S1 + S0 → S2　　（计算类）
I2: load (S2) → S3　　（访存类，「根据 S2 这个寄存器访问内存」）
I3: BEQ ……　　（转移类，「EQ 是个英文（词），是相等」）
I4: T2 + 20 → T2　　（I3 的下一条）
【逐步判】
I2 的 EX 在 I1 的 EX 之后即可（采用转发处理 I1→I2 的数据冒险）
I3 「既会用到 2 又会用到 3」——正常应等到 I1、I2 的 WB 之后；
　但要命的是 I2 是 load：「因为 load 的指令它在 M 阶段才会拿到数据，
　所以你的 EX 应该放在 M 之后……原因就是因为第二条是一条 load 的指令，
　这个就很关键。」→ **I3 阻塞**
I4 前面是转移指令：「因为转移指令书上是讲过，他在 EX 阶段只是算了一下目的地址，
　在 M 阶段才会把这个转移的目的地址写回到 PC……所以 IF 要放在 M 之后」→ **I4 阻塞**
结论：「那这个第一条第二条都是正常执行，第三条阻塞了，因为 EX 阶段停了……
阻塞的指令只有 3 和 4。」　　[02:29:20]–[02:33:40]

他评价这道题：「这个题目考得很综合，就基本就把刚才学过的东西都用到了……把它当做学完知识点、考完之后现在把它搞懂就可以了」（ [02:32:48]–[02:33:02] ）。

4.5 两道课后题（他用来练「判冒险」）

【课后题 25】问哪两条指令之间存在数据冒险
「前面是第一条是写回 1，第二条写回 5，那第三条读 5，刚好二和三是（有）数据冒险的……
真正有数据冒险的就是 2 和 3。」　　[02:34:19]–[02:35:07]
【课后题 29】问哪两条「不存在」数据冒险
「第一条是写回 s2，第三条是读 rs2……中间间隔了一条，所以是有数据冒险的；
然后第二条是写回三，第三条是读三，写后读，而且他们中间没有间隔，也是有数据冒险；
再看二和四：二是写回三、四是读二，这个就没关系……」　　[02:36:51]–[02:37:36]
★ 他借这两道题重申纪律：「做题目千万不要仅仅通过写后读来判断，
还要看一下他们中间间隔了几条。」　　[02:36:31]–[02:36:38]

5 统考真题与年份认定依据

年份怎么认定的（照实说）

本讲的年份**全部来自老师本人口播**，没有一处是我按原卷推断出来的。他这一讲总共说出了**六个年份**：2009、2012、2014、2015、2022、2023。其中三处我又用公开试题做了**外部核对**（2009、2012、2022，见下表最后一列），核对结果与他的口述一致；**核对只用来确认年份与题面措辞，不用来增添他没讲的内容**。凡他只说课本题号（24、25、28、29）的，本资料就照写「课后题」，不替它补年份。

题目	年份	他说的原话（含时间戳）	外部核对
数据通路大题（§4.1）	2009	「看下 09 年这个真（题）的图」`[00:21:51]`；「可以把书上的第九、第九年这道大题可以看一下」`[00:27:01]`；「看一下 09 年的（大）题」`[00:32:10]`	公开题面原文：`ADD (R1), R0` 的功能为 (R0)+((R1))→(R1)；两条 in 信号分别「允许数据从 DB 打入 MDR」（MDRinE）与「允许数据从内总线打入 MDR」（MDRin）——与他的口述一致 ✓
数据通路大题（§4.2）	2022	「看一下这个**二二年**（真）题」`[00:59:59]`；「整个的这个 22 年这个题目也都还好」`[01:23:37]`	题面（GPRs、FR 标志寄存器、暂存器、rs/rd 各 4 位、地址译码器 vs 多路选择器、控制信号由 CU 产生、IR 与 FR 连到控制部件输入端）与公开的 2022 年题一致 ✓
流水线大题（§4.3）	2012	「可以先看一下先看一下**一二年**的（那道）题」`[02:39:25]`、`[02:39:33]`	题面（16 位机、补码、`int` 型 −513 执行 `SHR R1`、连续 4 条指令进入流水线）与公开的 2012 年题一致 ✓
流水线选择题（§4.4）	2023	「这是**二三年**，二三年专门考过一个真（题）的选择题，叫 `load-use`」`[02:13:32]`；「它是**二、三年**的一个真（题）」`[02:26:59]`	未做外部核对（只按他口述）
本章第 4、5 题（cache，最后一小问才是流水线）	2014	「因为它也是 14 年的，两道题的题干是相关的，所以他就把它放到了一块，所以本质上还是 cache」`[02:39:02]`–`[02:39:12]`；「第四题和这个第五题，他们两个都是**一四年**的」`[03:12:27]`	未逐字核对；他说这一小问「到了最后一小问才是流水线」
15 年两道（数据通路 + 4.2 小节的指令）	2015	「之前也说过 **15 年**两道大题得放到一块；4.2 小节还有一道 15 年的，两道放一块去做」`[01:24:12]`–`[01:24:15]`；「一个是一五年的两道数据通路的题目——数据通路和指令」`[03:13:14]`	本讲**没有讲**这两道（只让课下做）
2023 年这道题的取材	不是年份	「因为这个王道书他也是根据——这一年考过这个题目之后，他才把那些东西都补充上去的」`[02:24:01]`：说明这道选择题**已被收入教材的例题/课后题**	—

容易被误当成年份的地方：① 他两次说的「三人一体」`[02:24:09]`（转写含糊，疑为「23 年题」或「三、四题」，**本资料不认定**）；② 课后题号 `24`、`25`、`28`、`29` 与书上页码 `205`、`213`、`220`、`254`、`255`、`262` 都是编号不是年份。

6 易错点清单

坑 / 错误	老师的说法与正确做法
把 MDRin 当成「从内存来」的那个信号	「MDRin 指的是从内部总线过来的，不要搞混了，所以这个 in 一定不能写」——2009 年题里从内存侧过来的是 `MDRinE`；2022 年题只给了内部总线这一组（`[01:08:16]`–`[01:08:51]`、`[00:33:06]`）
只写微操作、不写控制信号	「做题目的时候一定要审题……人家让写的是控制信号，不要写这种（微）操作」（`[01:06:52]`）
同一时钟里安排了两步都走内部总线的操作	「单总线的话一个时钟只能传一个数据，这句话要注意，很关键的一句话」（`[00:18:57]`）——只有一条走 CPU 总线、一条走系统总线时才能合并（`[00:46:03]`）
让 ALU 存数据 / 忘了先暂存	「ALU 这个硬件是没有存储数据的功能的」；运算前先把一个数放暂存器、运算结果先放暂存器或 ACC（`[00:42:01]`、`[00:41:55]`）
访存不经过 MAR / MDR	「凡是和内存打交道，必须借助 MAR 和 MDR」（`[00:43:09]`）；「凡是从内存拿数据你必须先把这个地址送到 MAR」（`[00:40:56]`）
只看「写后读」就断言有数据冒险	「并不是说写后读一定是有数据冒险的……中间间隔的指令刚好是三个那就没问题；如果小于三个……如果大于等于三那就没有」（`[02:35:55]`–`[02:36:14]`）
前面是 load 时仍只保证「EX 在 EX 之后」	「`load` 的指令它在 M 阶段才会拿到数据，所以你的 EX 应该放在 M 之后」——这叫延迟加转发（`[02:31:27]`–`[02:31:35]`）
上一条是转移指令时把 IF 提到 MEM 之前	「下一条指令的 IF 一定要在上一条——如果上一条是转移指令——一定要在转移指令的 MEM 之后」（`[02:21:46]`）
算时钟周期时忘掉流水段寄存器的延迟	「每一个阶段还要加上一个 20……最后时钟周期就应该选这个最大的，然后加上这个流水段寄存器，就是 100」（`[01:47:53]`–`[01:48:28]`）
问「至少」却只写一种方案	「他说至少，一般可能是有多种情况，你写的可能就不是那个至少的情况」——18 与 17 之差就是分（`[03:03:27]`–`[03:03:30]`、`[03:11:14]`）
16 位寄存器里塞 32 位 int 却按 32 位算	「正常来说这个 `int` 的话它是一个 32 位……你把它放到寄存器，他只会保留（低）16 位」（`[02:47:01]`–`[02:47:07]`）

⚠ **他本讲没有讲的易错点**（不要到这里找）：超标量与动态调度、分支预测（静态/动态）、延迟槽、流水线的吞吐率 / 加速比 / 效率——**这些词在全篇讲稿里一次都没有出现**（详见 §附）。

7 考场速查卡

【一】取指阶段模板（数据通路题的第一段几乎都能套）
(PC)→MAR → M(MAR)→MDR → (MDR)→IR → (PC)+1→PC
控制信号：PCout、MARin、读信号、（MDRinE 仅在给了内存侧信号时写）、MDRout、IRin
记住：in=写入、out=输出；单总线一个时钟只传一个数据；访存必经 MAR 和 MDR
【二】数据通路题的三个判断
① ALU 不存储 → 运算前先暂存一个操作数，运算结果先落暂存器/ACC，再上总线
② 两步能不能合并同一时钟 → 看是不是走了两条不同的总线（CPU 总线 vs 系统总线）
③ 题目问「最少时钟」→ 必须用合并后的写法；没问 → 两种写法都对
【三】五阶段流水线
阶段：IF 取指 | ID 译码+读寄存器 | EX 执行 | MEM 访存 | WB 写回
时间：K 级流水线执行 N 条指令 = (K + N − 1)T；问「周期数」就不乘 T
每个流水段后面有【流水段寄存器】（锁存本段数据，供下一段使用）
时钟周期 = 最长阶段时间 + 流水段寄存器延迟（课后题 28：取最大后 +20 = 100）

【四】三类指令在 EX / MEM 的差别（这张表必须背）
计算类：EX 运算　　MEM 空　　WB 写寄存器
访存类：EX 算有效地址　MEM 访存　　WB 写寄存器（读出的数据）
转移类：EX 算目的地址　MEM 写回 PC
【五】三条时序约束（画流水线图按顺序卡）
未用转发 + 写后读 → ID 在 WB 之后
用了转发　　→ EX 在 EX 之后
上一条是 load　→ EX 在 MEM 之后（延迟+转发）
上一条是转移指令 → IF 在 MEM 之后；任何情况下 IF 都不早于上一条的 ID
【六】三类冒险速判
结构冒险 = 资源冲突：同一时刻用同一部件（IF 访存 撞 MEM 访存）
　→ 排图时推迟 IF；根本解法：指令 cache 与数据 cache 分离
数据冒险 = 写后读：后面要用前面还没写回的结果
　→ 硬件阻塞 / 插入空操作 / 数据旁路（转发）/ load-use 用延迟+转发
控制冒险：转移指令改变执行流，后面可能白执行
　→ 尽早算出转移目标地址；IF 排到转移指令的 MEM 之后
★ 判数据冒险不能只看写后读：中间隔 ≥3 条指令就没有冒险

附：本资料的来源与边界

来源	内容	可靠性与边界
B 站《408 各科大题知识点强化课》第 8 讲「组成原理 · CPU 数据通路和流水线」，分 P10，**时长 3 小时 14 分 26 秒**	§1–§7 的全部	本地 ASR 转写（同批讲稿口径为 Whisper）。讲稿 `.md` 共 **2448 行 / 125 848 字节**，带时间戳的行 **2446 条**，覆盖 `[00:00:06]`–`[03:14:02]`。**其中 `[01:26:14]`–`[01:37:38]` 约 11 分钟是课间休息**（只有「中间休息一下，待会接着看这个流水线」一句）。**他上课时是当场画图板书的**（本讲的流水线时空图、数据通路连线全在纸上），转写稿里只有口述——所有时序图、连线与控制信号都是按他念出的先后关系还原的，凡还原处都标了 ⚠️。
公开试题的题面核对（**只用于确认年份与措辞**）	§4.1 的 `ADD (R1), R0` 与 `MDRin/MDRinE`；§4.2 的 GPRs/FR、暂存器、rs/rd 4 位、地址译码器、CU 输入端；§4.3 的 `SHR R1` 与 −513	核对的是**题面怎么写**，不是**老师讲了什么**。核对来源：`https://csgraduates.com/study_methods/408quiz/2009/`（2009 年）、`https://zhuanlan.zhihu.com/p/659130642`（2022 年）、2012/2014 年真题公开 PDF 与解析页。**本资料没有采用任何一条老师没讲过的新知识点**，除下面「教材补充」那一行。
教材补充（明确标注「教材补充」的地方）	① §2.1 表里 `TR` 这个缩写；② §3.1 注里「2022 年题面把两个暂存器写成 Y 和 Z」；③ §7 里「时钟周期 = 最长阶段时间 + 流水段寄存器延迟」这个**写成公式的形式**（他只说「选最大的，加上流水段寄存器，就是 100」）	**都不是他的原话**：本讲全文检索**没有**「吞吐率」「加速比」「效率」「流水线周期」「分支预测」「延迟槽」「超标量」「动态调度」中的任何一个词（各 0 次）。他给的与时间有关的公式**只有 (K+N−1)T 一个**。

⚠ **转写含糊、按上下文推断的地方（①–⑧）**

① `[00:01:16]`「流水线是三简易……这个简易是因为有一道是**开始的**」：按他讲 14 年题时说的「开始相关的……问的都是典型的开始一些东西，**命中率**啊之类的」，认定**「开始」= cache**；「简易」二字转写不通，本资料只保留可确证的语义（流水线部分的真题中含一道 cache 题）——**属推断**。

② `[00:32:56]`「MDR 印役」与 `[00:37:57]`「一个是往内存方向是 out1」：按 2009 年题面应为 `MDRinE` 与（往内存方向的）`MDRoutE`；转写没有念清助记符，本资料只保留方向语义。

③ `[00:36:44]` 起他给执行阶段标的节拍号（「C5」「第五步和第七步」「C7 C8 C9」）**转写不完整**：可以确定的是执行阶段原答案 6 拍、优化后 5 拍，本资料**不还原具体节拍编号**。

④ `[01:09:33]`「发出（读）命令到读出数据需要五个时钟周期」中的「5」是转写唯一给出的读内存延迟，与「寄存器间传数据 1 个时钟」「总共 7 个时钟周期」自洽（1+5+1=7），故采用。

⑤ `[02:47:15]`–`[02:51:54]` 第一问：他中途把 `FDFF` 写成 `FFDF`（并据此算出 `FFEF`），随后自己纠正回 `FDFF` → 右移得 `FEFF`。本资料采用**他纠正后的值**，并在正文里保留了他「算错了、这里都要改」的过程。

⑥ `[00:02:43]` 他念暂存器的缩写时转写完全糊（原文只剩「战存器一般是 t 用 t 表示」），本资料**不替他写这个缩写**。

⑦ `[02:24:09]`「可以做一下这个**三人一体**」、`[02:34:19]`「就是流水线这边考的 **25 题之所以就很简单了**」等处的题号/说法转写不通，本资料只采用能与他后续讲解对应的部分（25 题、29 题）。

⑧ §4.4 的四条指令里，寄存器名按转写照录为 `S1/S0/S2/S3/T2`，⚠ 疑为 `R` 系列寄存器被转写成 S/T；**不影响结论**（被阻塞的是第 3、4 条）。

老师明确说「本讲没讲 / 留到课下」的内容（别在这里找）

① **2009 年那道题只讲了执行阶段**：「但这个 09 年这个题就（先）不讲了……相对来说这个题目还是要简单一点，看答案应该就没有太大问题」（`[00:25:03]`–`[00:25:11]`）。

② **2015 年两道、2014 年两道共四道真题留到课下**：「相当有四道题可以留到课下」（`[03:13:11]`–`[03:13:25]`）；他承诺把单题讲解放进回放：「到时候我把那个相关的单个题目的回放……发在这个上课的回放里面」（`[03:12:35]`）。

③ **流水线的吞吐率 / 加速比 / 效率**：本讲完全没有出现（全篇 0 次），他只讲了 (K+N−1)T 与流水段寄存器延迟。

④ **分支预测（静态/动态）、延迟槽、超标量、动态调度**：本讲完全没有出现（全篇 0 次）；他解决控制冒险只给了「尽早算目标地址」与「IF 排在 MEM 之后」。

⑤ **一、二、三、六、七章的其余大题**：他只说「后面六七章的知识点已经会简单一些了」（`[03:13:35]`），这些留给后面的课。

⑥ **上一讲没讲完的题**：「当时就算应该有一两道题没讲」，靠之前发的资料与回放补（`[00:00:25]`–`[00:00:30]`）。

整合自 3 小时 14 分 26 秒网课讲稿（本地 ASR 转写；课堂上他当场画图，转写稿只有口述，时序图与控制信号按他口述的先后关系还原）· 未上传任何材料到第三方服务 · 生成于 2026-10-10